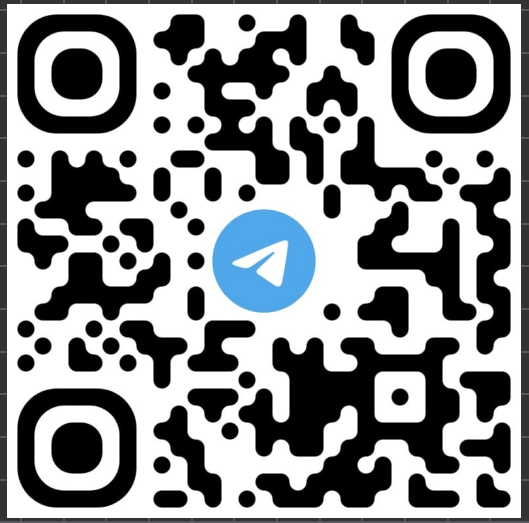
Telegram



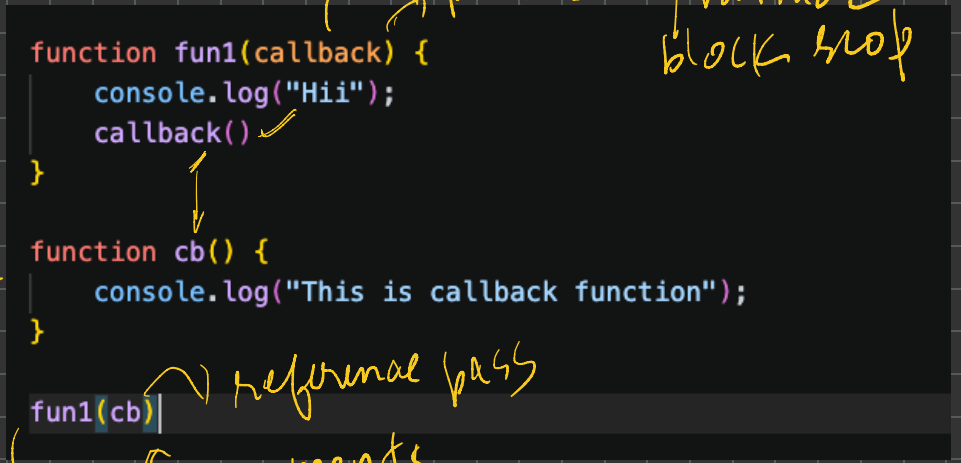
Discord



Callback & Callback Hell

Callback function

Jab tum kisi function ko as a argument pass krte ho kisi function call mai toh us fun. ko callback fun kehte hai


function fun1(callback) {
console.log("Hii");
callback()
}
function cb() {
console.log("This is callback function");
}
fun1(cb)


Callback
Synchronous
Asynchronous

High order function

→ Any function that takes a function as an argument.

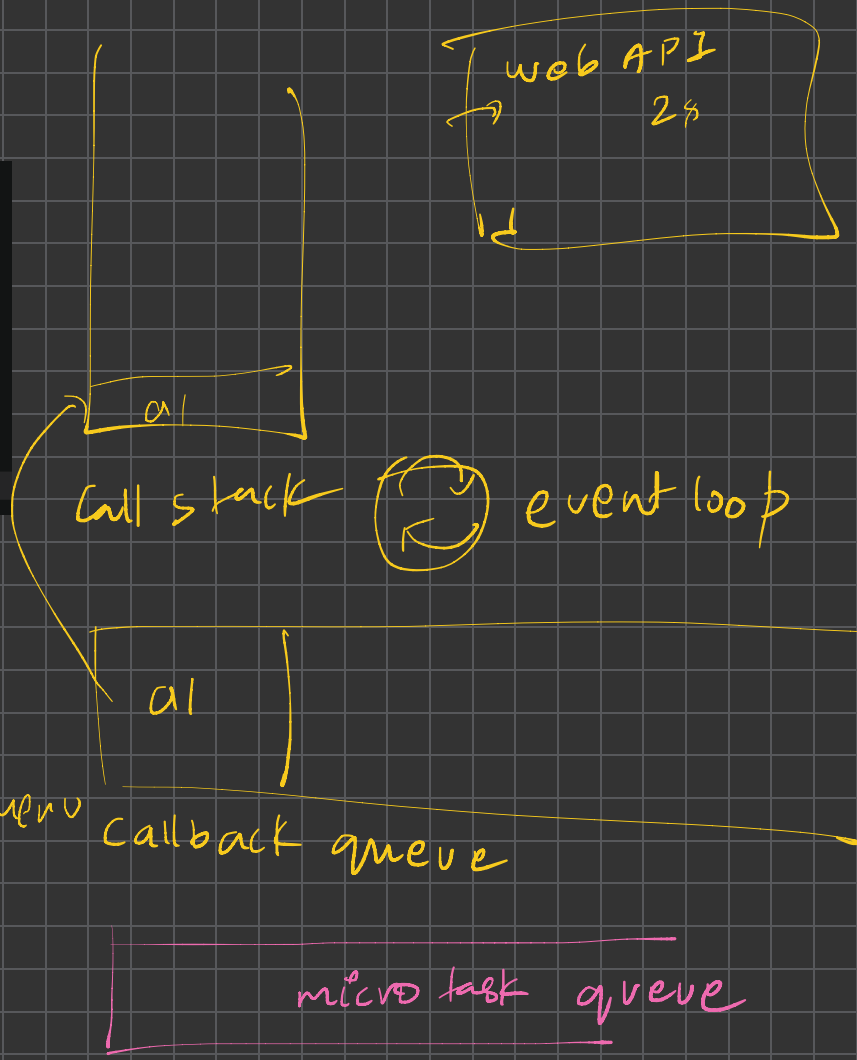
OR

→ Any function that Returns a function.

Example

```
function searchPizza(){
    console.log("Pizza searching...");
    setTimeout(function (){
        console.log("Here is the Pizza's Menu.");
        return 500;
    } , 2000)
}


let output = searchPizza()
console.log(output);
```



1. Pizza searching
2. undefined
3. Here is the Pizz Menu

```
function searchPizza(cb1){
    console.log("Pizza searching...");
    setTimeout(function (){
        console.log("Here is the Pizza's Menu.");
        let price  = 500;
        cb1(price)
    } , 2000)
}


searchPizza(function (price) {
    console.log(price);
})
```

① ② ③

closures

searchPizza
order Pizza
payment

searchPizza() → clg ① → ✗

func() {} → ② → cb1() → clg ③

```
function searchPizza(cb1){
    console.log("Pizza searching...");
    setTimeout(function (){
        console.log("Here is the Pizza's Menu.");
        let price  = 500;
        cb1(price)
    } , 2000)
}

function addToCart(cb2){
    console.log("Pizza adding to cart...");
    setTimeout(function(){
        console.log("Pizza Added to cart");
        cb2()
    } , 3000)
}

function paymet(price , cb3){
    console.log(`Payment Initiated , Amount : ${price}`);
    setTimeout(function (){
        console.log(`Payment Completed, Amount : ${price}`);
        cb3()
    } ,5000)
}

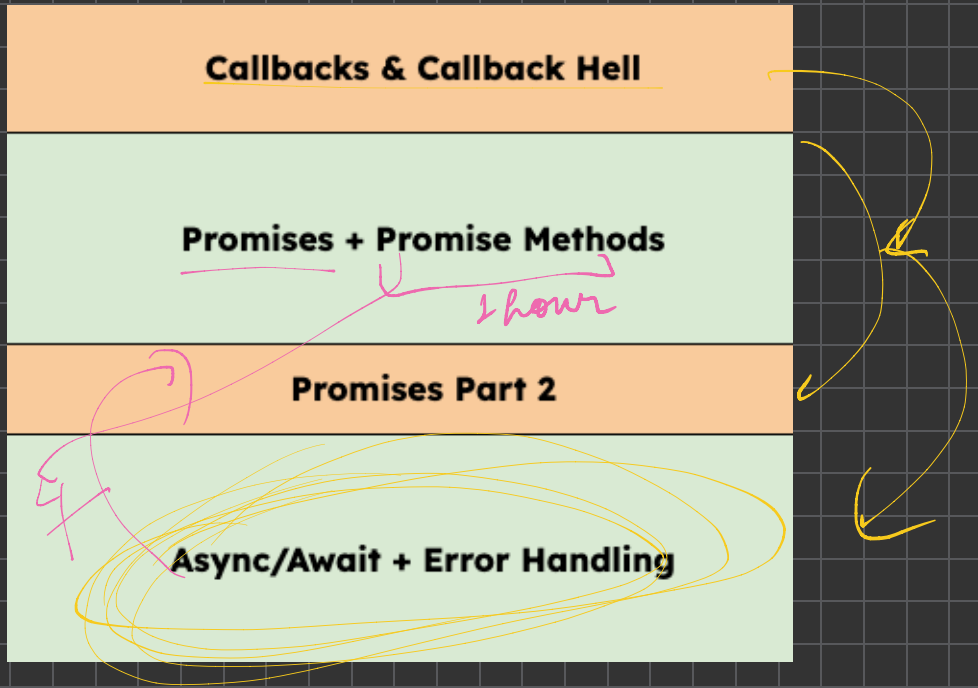

searchPizza(function (price) {
    addToCart(function () {
        paymet(price ,function () {
            console.log("Bss Aa hee gaya Pizza");
        })
    })
})
```

searchPizza()

addToCart

payment

searchPizza
addToCart
applyCoupon
addAddress
payment
sendNotification


Callbacks & Callback Hell
Promises + Promise Methods
1 hour
Promises Part 2
Async/Await + Error Handling

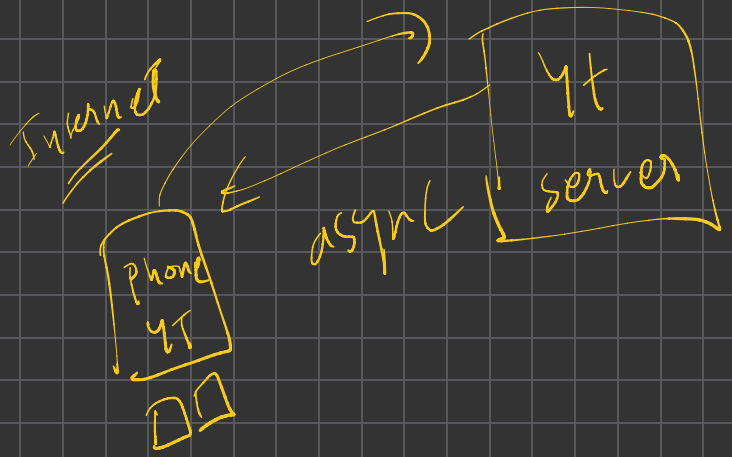
Pyramid

```
searchPizza(function a(price) {
    addToCart(function b() {
        paymet(price ,function c() {
            console.log("Bss Aa hee gaya Pizza");
        })
    })
})
```

```
// Callback Hell


a(function (resultsFromA) {
    b(resultsFromA, function (resultsFromB) {
        c(resultsFromB, function (resultsFromC) {
            d(resultsFromC, function (resultsFromD) {
                e(resultsFromD, function (resultsFromE) {
                    f(resultsFromE, function (resultsFromF) {
                        console.log(resultsFromF);
                    })
                })
            })
        })
    })
});

```


Internet
Phone
YT
async
server
YT

```
function searchPizza(cb1){
    console.log("Pizza searching....")
    setTimeout(function fun1(){
        console.log("Here is the pizza's Menu");
        let price= 699;
        cb1(price);

    } ,2000);
}

function addToCart(cb2){
    console.log("pizza add to cart");
    setTimeout(function fun2(){
        console.log("Pizza add to cart");
        cb2();
    },3000);

}

function payment(price ,cb3){
    console.log(`Payment Initiated,Amount : ${price}`);
        setTimeout(function fun3(){
            console.log(`Payment Completed,Amount: ${price}`);
            cb3();

        },5000);

}
searchPizza(function(price){
    addToCart(function  a (price) {
        addToCart(function b (){
            payment(price,function c (){
                console.log("Pizza Deliverd");


            })
        })
    })
```

Inversion of control

↳ we give control of something to someone else.

```
searchPizza(function a(price) {
    addToCart(function b() {
        paymet(price)
        paymet(price)
    })
})
```

} → inversion of control